

MODULE *hashes_hints*

get-hashes scheduling-hint verification specification.

get-hashes returns per-block sync-cost metadata — hash, size, *txs*, *txouts*, notes — as scheduling hints. The draft’s deferred-penalty rules (“get-hashes”, “Misbehavior and Banning”):

- *txs* and *notes* are determined by the block hash alone, so once the block is downloaded a mismatch proves the *HINT* server lied : *SHOULD* penalize(20 points).
- *size* is *NOT* determined by the hash : a block’s serialized size depends on authorizing data the txid does not commit to. A node that got hints from one peer and blocks from another *MUST NOT* penalize a size mismatch unless it has verified the delivered block’s authorizing data commitment (which binds the serialization via the header); “otherwise a peer that pads the blocks it serves would cause the penalty to fall on the honest peer that served the hints, which is an eviction primitive rather than a defense.”

The model: a requester takes hints for one block from a hint server *HS* and the block itself from a block server *BS*. *HS* may lie about *txs* or size

(*ByzHints*); *BS* may pad the block’s serialization without changing its hash

(*ByzBlocks*). The requester checks hints on download and may verify the authorizing data commitment afterwards. The *PenalizeSizeUnverified* switch is the forbidden reading. *PenaltyImpliesLie* is the provability principle at this corner: every penalty lands on a peer that actually lied — and it is violated exactly when a padding *BS* frames an honest *HS* through an unverified size mismatch, the draft’s own eviction-primitive warning reproduced as a counterexample. *LiarEventuallyPenalized* checks the deferred penalty does land on a lying hint server.

Also noted while modeling (feedback item 14): *txouts* is determined by the block hash exactly as *txs* is, and the draft has the requester rely on it(the spentness – hint bitmap), yet it appears in neither the *SHOULD* – verify sentence nor the penalty table.

See .. /documents/v2 – modeling.md for the full write – up.

EXTENDS *Naturals*, *FiniteSets*

CONSTANT <i>ByzHints</i>	the hint server may lie about <i>txs</i> or size
CONSTANT <i>ByzBlocks</i>	the block server may pad the serialization
CONSTANT <i>PenalizeSizeUnverified</i>	TRUE penalizes a size mismatch before auth verification

TrueTxs $\triangleq$ 2 ground truth for the one block modeled
TrueSize $\triangleq$ 5
PaddedSize $\triangleq$ 7

VARIABLES

<i>hint_txs</i> ,	hint received from <i>HS</i> (0 = not yet received)
<i>hint_size</i> ,	
<i>blk_size</i> ,	serialized size of the delivered block(0 = not yet delivered)
<i>checked</i> ,	the on – download hint check has run
<i>verified</i> ,	the authorizing data commitment has been checked
<i>hs_score</i> ,	penalty assigned to the hint server
<i>hs_lied</i>	ghost : <i>HS</i> actually lied

vars $\triangleq$ $\langle$ *hint_txs*, *hint_size*, *blk_size*, *checked*, *verified*, *hs_score*, *hs_lied* $\rangle$

Init $\triangleq$

$\wedge \text{hint_txs} = 0 \wedge \text{hint_size} = 0$
 $\wedge \text{blk_size} = 0$
 $\wedge \text{checked} = \text{FALSE}$
 $\wedge \text{verified} = \text{FALSE}$
 $\wedge \text{hs_score} = 0$
 $\wedge \text{hs_lied} = \text{FALSE}$

HS serves the hints; a Byzantine HS may misstate txs or size.

ServeHints $\triangleq$

$\wedge \text{hint_txs} = 0$
 $\wedge \exists t \in \text{IF } \text{ByzHints} \text{ THEN } \{ \text{TrueTxs}, \text{TrueTxs} + 1 \} \text{ ELSE } \{ \text{TrueTxs} \} :$
 $\quad \exists z \in \text{IF } \text{ByzHints} \text{ THEN } \{ \text{TrueSize}, \text{TrueSize} + 1 \} \text{ ELSE } \{ \text{TrueSize} \} :$
 $\quad \wedge \text{hint_txs}' = t$
 $\quad \wedge \text{hint_size}' = z$
 $\quad \wedge \text{hs_lied}' = (t \neq \text{TrueTxs} \vee z \neq \text{TrueSize})$
 $\wedge \text{UNCHANGED } \langle \text{blk_size}, \text{checked}, \text{verified}, \text{hs_score} \rangle$

BS delivers the block. Its transactions are determined by the hash (a different transaction set would not hash to the requested block), so txs is always the truth; the serialization is not, and a Byzantine BS may pad it.

ServeBlock $\triangleq$

$\wedge \text{hint_txs} \neq 0$
 $\wedge \text{blk_size} = 0$
 $\wedge \exists z \in \text{IF } \text{ByzBlocks} \text{ THEN } \{ \text{TrueSize}, \text{PaddedSize} \} \text{ ELSE } \{ \text{TrueSize} \} :$
 $\quad \text{blk_size}' = z$
 $\wedge \text{UNCHANGED } \langle \text{hint_txs}, \text{hint_size}, \text{checked}, \text{verified}, \text{hs_score}, \text{hs_lied} \rangle$

On download, the requester checks the hash – determined hints : a txs mismatch proves the hint server lied. A size mismatch proves nothing yet; penalizing it anyway is the forbidden switch.

CheckOnDownload $\triangleq$

$\wedge \text{blk_size} \neq 0$
 $\wedge \neg \text{checked}$
 $\wedge \text{checked}' = \text{TRUE}$
 $\wedge \text{hs_score}' = \text{hs_score}$
 $\quad + (\text{IF } \text{hint_txs} \neq \text{TrueTxs} \text{ THEN } 20 \text{ ELSE } 0)$
 $\quad + (\text{IF } \text{PenalizeSizeUnverified} \wedge \text{hint_size} \neq \text{blk_size} \text{ THEN } 20 \text{ ELSE } 0)$
 $\wedge \text{UNCHANGED } \langle \text{hint_txs}, \text{hint_size}, \text{blk_size}, \text{verified}, \text{hs_lied} \rangle$

The requester verifies the authorizing data commitment : the canonical serialization | and so the true size | is now bound to the hash, and a hint that misstated it is provably the hint server's lie.

$$\begin{aligned}
\textit{VerifyAuthData} &\triangleq \\
&\wedge \textit{blk_size} \neq 0 \\
&\wedge \textit{checked} \\
&\wedge \neg \textit{verified} \\
&\wedge \textit{verified}' = \text{TRUE} \\
&\wedge \textit{hs_score}' = \textit{hs_score} + (\text{IF } \textit{hint_size} \neq \textit{TrueSize} \text{ THEN } 20 \text{ ELSE } 0) \\
&\wedge \text{UNCHANGED } \langle \textit{hint_txs}, \textit{hint_size}, \textit{blk_size}, \textit{checked}, \textit{hs_lied} \rangle
\end{aligned}$$

$$\textit{Next} \triangleq \textit{ServeHints} \vee \textit{ServeBlock} \vee \textit{CheckOnDownload} \vee \textit{VerifyAuthData}$$

Every step of the flow is weakly fair; each fires once.

$$\begin{aligned}
\textit{Spec} &\triangleq \textit{Init} \wedge \Box[\textit{Next}]_{\textit{vars}} \wedge \text{WF}_{\textit{vars}}(\textit{ServeHints}) \wedge \text{WF}_{\textit{vars}}(\textit{ServeBlock}) \\
&\wedge \text{WF}_{\textit{vars}}(\textit{VerifyAuthData}) \wedge \text{WF}_{\textit{vars}}(\textit{CheckOnDownload})
\end{aligned}$$

Liveness: a lying hint server is eventually penalized (the deferred penalty lands).

$$\textit{LiarEventuallyPenalized} \triangleq (\textit{hs_lied} \leadsto \textit{hs_score} > 0)$$

Safety invariants.

$$\begin{aligned}
\textit{TypeOK} &\triangleq \\
&\wedge \textit{hint_txs} \in \{0, \textit{TrueTxS}, \textit{TrueTxS} + 1\} \\
&\wedge \textit{hint_size} \in \{0, \textit{TrueSize}, \textit{TrueSize} + 1\} \\
&\wedge \textit{blk_size} \in \{0, \textit{TrueSize}, \textit{PaddedSize}\} \\
&\wedge \textit{hs_score} \in \{0, 20, 40\}
\end{aligned}$$

The provability principle at this corner: every penalty lands on a peer that actually lied. Violated when a padding block server frames an honest hint server through an unverified size mismatch.

$$\textit{PenaltyImpliesLie} \triangleq \textit{hs_score} > 0 \Rightarrow \textit{hs_lied}$$
